제24회 임베디드SW경진대회

개발완료보고서

부 문	□ 자유공모 □ 스마트 가전 ■ 자동차/모빌리티
작 품 명	인지기능 저하 운전자를 위한 멀티모달 상황 해석 기반 운전 보조 시스템
팀 명	CODRIVE
팀 원	서유나(팀장), 권현준, 백건하, 유용준, 황예은
소 속	건국대학교
제 출 일	2026.10 .28 .

목 차

1	개발 개요	
2	개발 환경 설명	
3	개발 프로그램 설명	
4	개발 중 발생한 장애요인과 해결방안	
5	개발 결과물의 차별성	
6	개발 결과물의 파급력 및 기대효과	
7	개발 일정 및 업무 분장	

1. 개발 개요

작품명	인지기능 저하 운전자를 위한 멀티모달 상황 해석 기반 운전 보조 시스템
개발 배경	최근 자동차에는 전방 충돌 경고, 차선 이탈 경고, 운전자 상태 감지 등 다양한 첨단 운전자 보조 시스템(ADAS)이 적용되고 있다. 그러나 실제 주행 환경에서는 전방 보행자 접근, 차선 이탈, 운전자 전방 미주시 등 여러 위험 요소가 동시에 발생할 수 있다. 이때 개별 위험에 대한 경고가 중복 제공되면 운전자는 각 경고의 중요도를 직접 판단해야 하므로 인지적 부담이 증가할 수 있다. 특히 고령 운전자의 증가에 따라 운전 중 위험 상황을 직관적으로 이해하고 대응할 수 있도록 지원하는 기술의 필요성이 커지고 있다. 국가데이터처의 「2025 고령자 통계」에 따르면 국내 65세 이상 인구는 전체 인구의 20.3%를 차지한다. 또한 Euro NCAP의 2026년 안전 평가 체계에서도 운전자 상태 모니터링과 운전 보조 기능의 연계, HMI 사용성이 강조되고 있어 운전자 중심의 통합 안전 기술이 중요해지고 있음을 보여준다. 이에 CODRIVE는 차량 외부 영상, 내부 운전자 영상, 음향 정보 및 CAN 기반 차량 상태를 종합적으로 분석하여 복합 위험 상황을 판단하는 운전 보조 시스템을 개발하였다. NVIDIA Jetson, STM32, Raspberry Pi 5로 구성된 분산형 임베디드 시스템에서 각 정보를 수집하고, Situation Snapshot 기반 데이터 통합과 위험도 분석을 통해 가장 시급한 위험을 선정한다. 이를 바탕으로 운전자에게 여러 경고를 단순 나열하는 대신 **최우선 행동지침을 직관적으로 제공하여 주행 중 인지 부담을 줄이는 것**을 목적으로 한다.
개발 동기	기존 ADAS는 객체나 이벤트의 존재를 알려주는 기능에는 효과적이지만, 여러 위험이 동시에 발생했을 때 운전자에게 어떤 행동을 우선적으로 수행해야 하는지까지 판단하여 제공하는 데에는 한계가 있다. 예를 들어 동일한 거리의 보행자가 전방에 존재하더라도 운전자가 정상적으로 전방을 주시하고 있는 경우와 휴대폰을 사용하며 전방을 보지 않는 경우의 실제 위험 수준은 다르다. 따라서 단순히 외부 객체를 인식하는 것에서 나아가 차량 외부 영상, 내부 운전자 영상, 음향 정보, 차량 상태를 하나의 Situation Snapshot으로 통합하고, 각 위험 요소의 중요도를 계산하여 운전자에게 가장 필요한 행동만을 전달하는 시스템이 필요하다고 판단하였다.
개발 목표	본 작품은 다음과 같은 기능 구현을 목표로 개발하였다. ● 외부 카메라 기반 차량, 보행자, 신호등 및 차선 인식 ● 내부 카메라 기반 운전자 졸음, 휴대폰 사용 및 정면 응시 상태 인식 ● 외부 음향 센서를 이용한 긴급차량 사이렌 및 경적 인식 ● Logitech G29을 이용한 조향 및 브레이크 등 차량 입력 생성 ● CANable V2.0을 이용한 가상 차량 CAN 메시지 송신 ● STM32 NUCLEO-H723ZG 기반 차량 CAN 데이터 수신 및 디코딩 ● ROS 2 DDS를 이용한 임베디드 장치 간 데이터 송수신 ● 각 센서 및 차량 데이터를 Situation Snapshot으로 통합 ● XGBoost 모델 기반 위험도 판단 ● 운전자 HMI 구현 최종적으로 특정 시연 시나리오에 한정되지 않고 다양한 주행 환경에서도 사용할 수 있는 확장 가능한 위험 인식 및 경고 시스템을 구현하는 것을 목표로 하였다.

필요성	도로에서는 하나의 위험 요소만 독립적으로 발생하지 않는다. 차량, 보행자, 신호등, 차선 등의 외부 환경과 함께 운전자의 상태 및 차량의 움직임이 동시에 실제 위험 수준에 영향을 미친다. 그러나 개별 센서 기반 시스템에서는 각각의 정보를 별도 판단하기 때문에 이러한 복합적인 상황을 충분히 반영하기 어렵다. 또한 여러 위험이 동시에 검출되는 경우 모든 경고를 운전자에게 제공하면 정보량이 증가하여 실제로 가장 긴급하게 대응해야 하는 위험을 빠르게 판단하기 어려울 수 있다. CODRIVE는 다중 센서 정보를 하나의 Situation Snapshot으로 통합하여 위험도를 종합적으로 판단하고, 위험의 심각도와 긴급성을 기반으로 경고 우선순위를 선정한다. 이를 통해 운전자가 현재 가장 중요한 위험을 빠르게 인지할 수 있도록 지원한다.
소스코드 링크	https://github.com/skysky0214/2026ESWContest_mobility_CODRIVE.git
시연 동영상 링크	https://youtu.be/mQxMPxAprZU

2. 개발 환경 설명

가. HW 구성 및 기능				
No	구성 요소	모델명	주요 사양	기능 및 역할
1	AI 연산 보드	NVIDIA Jetson	NVIDIA GPU 기반 Embedded AI Platform	외부 카메라 및 음향 기반 차량·보행자·신호등·차선·사이렌 인식
2	AI 연산 보드	NVIDIA Jetson	NVIDIA GPU 기반 Embedded AI Platform	내부 카메라 기반 운전자 졸음·휴대폰 사용·전방 주시 상태 인식
3	MCU	STM32 NUCLEO-H723ZG	STM32H723, FDCAN 지원	CAN 메시지 수신 및 차량 속도·브레이크·조향각 데이터 처리`
4	차량 입력 장치	Logitech G29	Steering Wheel / Pedal	조향·브레이크 입력을 가상 데이터로 생성 / CARLA 제어값으로 전달
5	CAN Interface	CANable V2.0	CAN Interface	PC에서 생성된 차량 CAN 메시지 송신
6	CAN Transceiver	SN65HVD230	CAN Transceiver	CAN 신호의 물리계층 송수신
7	중앙 판단 장치	Raspberry Pi 5	ARM 기반 SBC	Situation Snapshot 생성, Rule-based·XGBoost 위험도 판단, HMI 출력

HW 구성도

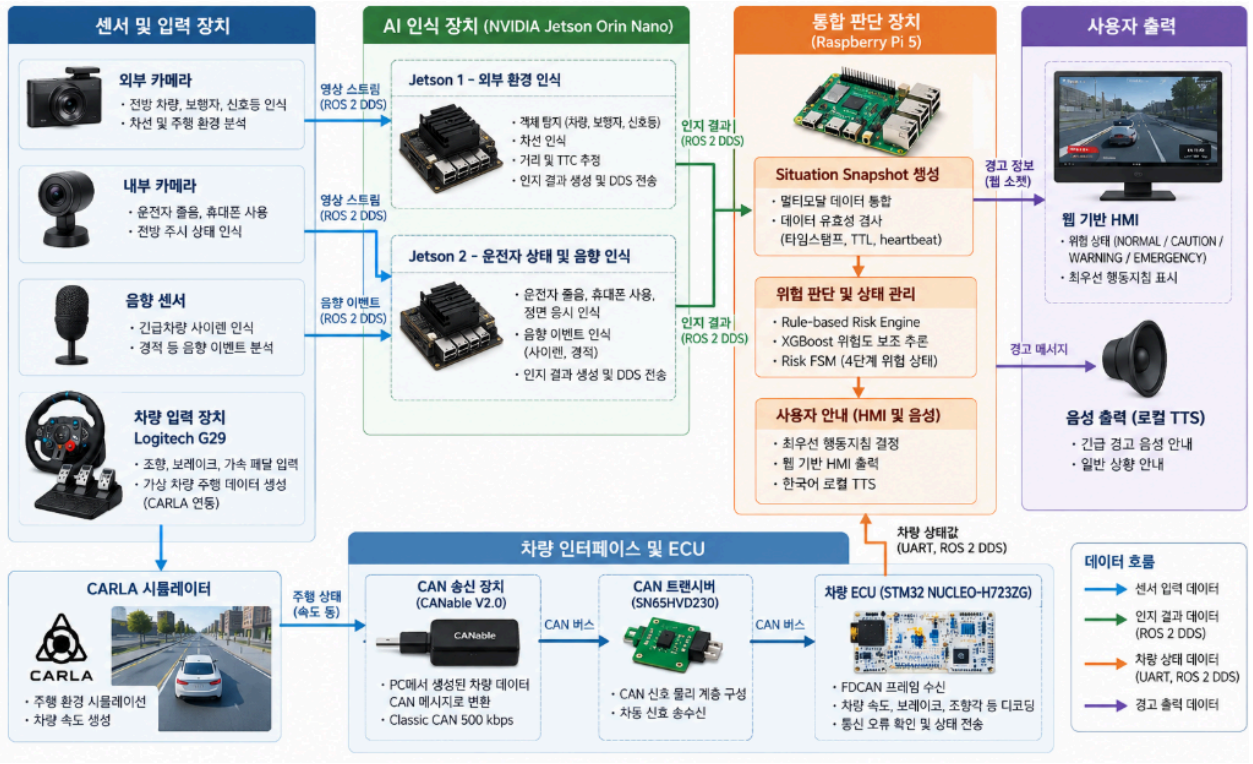


그림 2-1. HW 구성도

나. SW 구성			
No	구분	기술 및 도구	용도
1	OS 및 언어	Ubuntu 24.04, Windows, Python, C	보드 런타임 및 ECU 펌웨어
2	MCU 개발	STM32CubeIDE, STM32 HAL FDCAN	차량 CAN 프레임 수신 및 상태 처리
3	임베디드 통신	ROS 2, CycloneDDS	보드 간 데이터 전송 및 DDS 계약
4	인지 SW	YOLO, OpenCV, PyTorch	영상과 운전자 상태 인식
5	위험 판단	Rule-based Risk Engine, XGBoost	위험 후보와 보조 예측
6	표시 및 안내	웹 기반 HMI, 한국어 로컬 TTS	시각적 안내와 음성 출력
7	통합 및 배포	서비스 런처, 배포 manifest, Git	보드별 구동과 이력 관리
8	차량 시뮬레이터	CARLA Simulator	도로 환경 및 차량 주행 시뮬레이션
9	운전자 입력	Logitech G29 Interface	조향·가속·브레이크 입력을 CARLA Vehicle Control로 변환

다. 개발 환경	
운영체제 (OS)	Ubuntu 24.04, Windows
개발 언어	Python. C
프레임워크·라이브러리	ROS 2, PyTorch, YOLO, OpenCV, XGBoost, NumPy, python-can, PyQt5, CARLA Python API
개발 도구	Visual Studio Code, STM32CubeIDE, GitHub
통신·인터페이스	ROS 2 DDS, Classic CAN 500 kbps, CANable V2.0, STM32. FDCAN, Ethernet
형상 관리	GitHub, Feature Branch 기반 개발 및 버전 관리

라. 기능 설계도

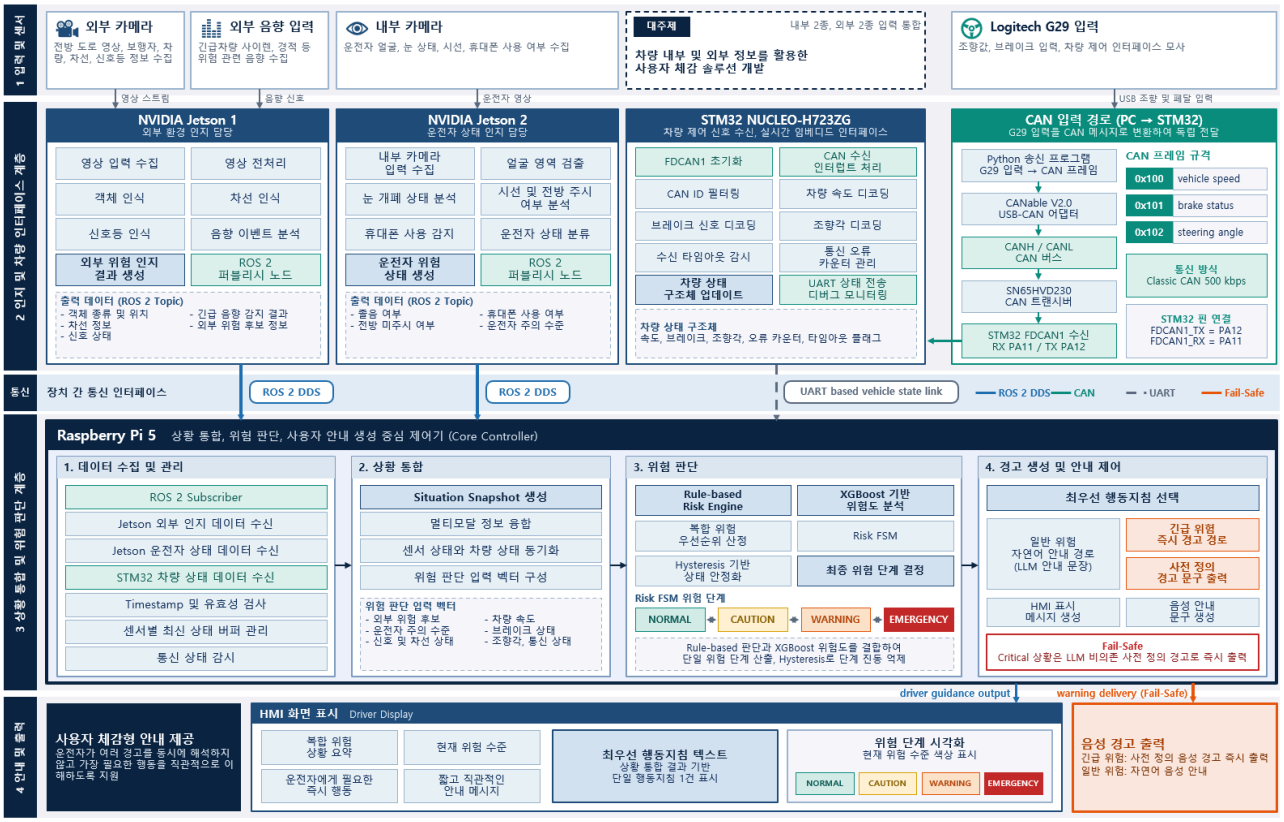


그림 2-2. 기능 설계도

3. 개발 프로그램 설명

가. 파일 구성		
No	파일 및 경로	설명
1	stm32/CODRIVE_ECU/Core/Src/main.c	STM32 FDCAN 수신 및 차량 상태 디코딩
2	tools/carla_can_sender.py	CARLA 주행 정보 및 G29 입력의 CAN 송신
3	board_runtime/src/codrive/apps/can_ecu.py	STM32 UART 차량 정보의 Raspberry Pi 연계
4	board_runtime/src/codrive/hardware/stm32_can.py	차량 데이터 파싱 및 유효성 검사
5	board_runtime/src/codrive/fusion.py	Situation Snapshot 기반 데이터 통합
6	board_runtime/src/codrive/live_risk_rules.py	실시간 차량 위험 규칙 처리
7	board_runtime/src/codrive/vehicle_dynamics.py	실시간 차량 위험 규칙 처리
8	board_runtime/src/codrive/risk_ml/	속도 변화 기반 가속도 추정
9	board_runtime/src/codrive/risk.py	위험도 판단 및 위험 상태 관리
10	tools/calibrate_front_camera.py	XGBoost 기반 위험도 보조 추론
11	tools/carla_pedestrian_scenario.py	전방 카메라 렌즈 보정

나. 함수별 기능				
파일명	함수명	입력	출력	기능
front_perception.py	perception_from_frame()	전방 카메라 Frame, Header, Camera ID	ExternalPerception	YOLO 기반 차량·보행자·신호등 객체를 탐지하고 신호등 상태 및 차선 신뢰도를 계산하여 외부 환경 인식 결과를 생성
dms.py	observe()	내부 카메라 YOLO Detection, Capture Time	DriverDmsObservation	얼굴·눈 감김·하품·휴대폰·전방/비전방 주시 검출 결과를 이용해 운전자 졸음 및 주의분산 상태를 판단
audio/model.py	predict_probabilities()	음향 Log-Mel Feature	사이렌·경적 확률	학습된 음향 분류 모델을 이용하여 입력 음향의 사이렌·경적 발생 확률을 추론
g29_can_sender.py	encode_frames()	차량 속도, 브레이크 상태, 조향각	CAN Frame 3개	차량 상태를 CAN ID 0x100, 0x101, 0x102 규격의 8-byte CAN 메시지로 인코딩

g29_can_sender.py	decode_frames()	CAN Frame	속도·브레이크·조향각 Dictionary	CAN 메시지를 차량 속도, 브레이크 상태, 조향각의 물리값으로 디코딩하여 통신 데이터 확인
jetson_publisher.py	external()	ExternalPerception	DdsEnvelope	Jetson 1의 외부 환경 인식 결과를 ROS 2 DDS 기반 Topic으로 송신
jetson_publisher.py	driver()	DriverState	DdsEnvelope	Jetson 2에서 생성된 운전자 상태 정보를 DDS 메시지로 변환하여 Raspberry Pi로 송신
fusion.py	ingest()	외부 인식·운전자·음향·차량 Sensor Sample	bool	각 장치에서 수신한 센서 데이터를 종류 및 Timestamp 기준으로 저장하고 유효성을 검사
fusion.py	snapshot()	현재 Timestamp	SituationSnapshot	시간적으로 유효한 최신 외부 환경·운전자·음향·차량 데이터를 통합하여 Situation Snapshot 생성
engine.py	assess()	SituationSnapshot	RiskAssessment	신호등·운전자 상태·음향·차량 정보를 Rule-based로 평가하고 위험 후보의 점수 및 우선순위를 계산한 뒤 FSM을 적용하여 최종 위험 상태 결정

다. 흐름도

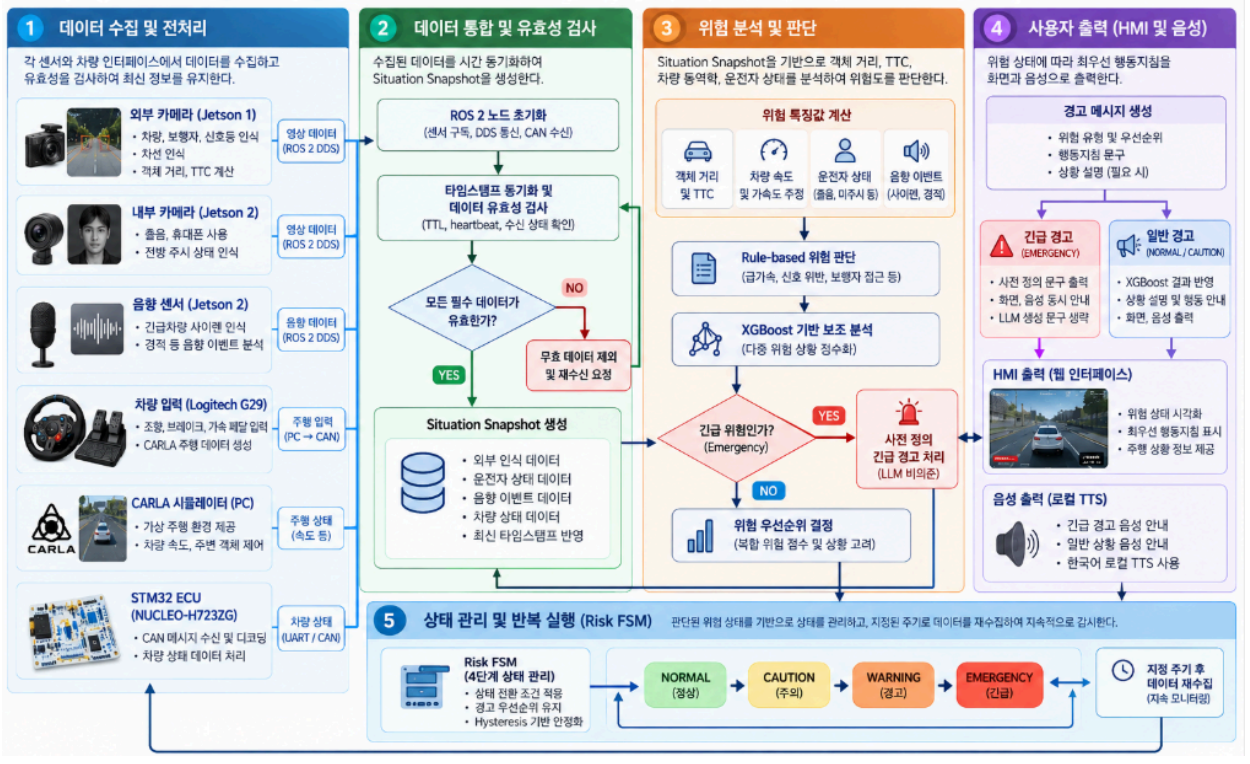


그림 3-1. 전체 동작 흐름도

라. 기술적 차별성	
①	**① 이기종 임베디드 장치 기반 분산형 소프트웨어 아키텍처** NVIDIA Jetson, STM32, Raspberry Pi 5의 하드웨어 특성에 따라 영상 인식, 차량 정보 처리, 위험 판단 및 경고 출력 기능을 분산 구현하였다. ROS 2 DDS 기반 데이터 통신과 타임스탬프, TTL, heartbeat를 활용한 유효성 관리를 적용하여 장치 간 데이터 정합성과 통신 신뢰성을 강화하였다.
②	**② CARLA와 실제 CAN 하드웨어를 연계한 차량 상태 처리** CARLA의 동적 주행 속도와 Logitech G29의 조향, 브레이크, 가속 페달 입력을 실제 CAN 프레임으로 변환하고, STM32 FDCAN에서 수신 및 디코딩하는 시스템을 구현하였다. 이를 통해 실차 없이도 실제 CAN 물리 계층과 MCU 펌웨어를 활용하여 차량 상태 처리 및 통신 오류 대응 기능을 검증할 수 있는 임베디드 시험 환경을 구축하였다.
③	**③ TTC 및 차량 동역학 기반 복합 위험 판단과 안전 경고** 외부 객체의 거리와 예상 충돌시간(TTC), 운전자 상태, CAN 기반 차량 속도 및 가속도 정보를 통합하여 복합 위험을 판단하도록 구현하였다. Rule-based Risk Engine과 XGBoost 기반 분석, Risk FSM을 활용하여 위험 상태와 경고 우선순위를 관리하며, 긴급 상황에서는 자연어 생성에 의존하지 않는 사전 정의 경고 경로를 통해 운전자에게 최우선 행동을 안내한다.

마. 실시간 위험 판단을 위한 모델 선정 및 경량화	
①	**① 초기 LLM 기반 판단 구조 및 한계** 초기 시스템에서는 Situation Snapshot으로 통합된 운전자 상태, 외부 환경 및 차량 정보를 해석하고 행동지침을 생성하기 위해 LLM 기반 접근을 검토하였다. 외부 LLM API를 이용한 초기 시험에서는 네트워크 요청 및 응답 과정과 모델 추론·문장 생성 과정에 따른 지연이 발생하였다. 본 시스템은 위험 상황 발생 후 운전자에게 경고가 전달되기까지의 종단 지연시간을 200ms 이내로 설정하였으며, 영상·음향 센서 처리, ROS 2 DDS 통신, 위험 판단 및 HMI 출력에 필요한 시간을 함께 고려해야 했다. 따라서 외부 API 의존 구조는 종단 지연 목표를 안정적으로 만족하기 어려울 가능성이 있다고 판단하여 로컬 경량 모델 및 비생성형 위험 판단 방식으로의 전환을 검토하였다.
②	**② 경량 언어모델 검토** 외부 API 사용으로 인한 지연을 줄이기 위해 소형 언어모델(SLM)의 로컬 실행 가능성을 검토하였다. Qwen3-0.6B, Qwen3-1.7B, Gemma 3 1B, Granite 4.0 350M 등을 후보로 선정하고, GGUF 기반 양자화 모델을 활용하는 방향을 검토하였다. 그러나 본 시스템의 실시간 경고 기능은 자유로운 문장 생성보다 위험 수준의 신속한 판단과 정해진 행동지침의 전달이 중요하다. 이에 따라 언어 생성 모델을 경고 경로의 필수 구성 요소로 사용하기보다, 구조화된 입력으로부터 위험도를 계산하는 경량 모델을 사용하는 방향으로 설계를 변경하였다.
③	**③ XGBoost 기반 위험 판단 구조** 최종적으로 Rule-based Risk Engine과 XGBoost 기반 위험도 보조 분석을 결합하고, Risk FSM을 통해 위험 상태와 경고 우선순위를 관리하는 구조를 채택하였다. • Rule-based Risk Engine: TTC, 차량 상태 및 명시적 위험 조건 분석

	• XGBoost: Situation Snapshot의 특징값을 이용한 위험도 보조 추론 • Risk FSM: 위험 상태 전이 및 Hysteresis 기반 상태 안정화 • HMI·TTS: 사전 정의된 경고 메시지 및 행동지침 출력 XGBoost 모델의 내부 추론시간은 팀 측정 결과 약 0.2ms로 확인되었다. 이는 모델 추론 단계의 측정 결과이며, 센서 처리와 통신, HMI 출력이 포함된 종단 지연시간과는 구분된다. 이러한 구조 변경을 통해 실시간 경고 경로에서 외부 LLM API 및 자연어 생성 결과를 필수적으로 기다리지 않도록 설계하였다.

4. 개발 중 발생한 장애요인과 해결방안

장애요인 1 – 가상 차량 데이터와 실제 ECU 통신 연계	
문제 현상	초기 CAN 통신 시험에서는 고정된 가상 속도값을 사용하여 차량의 실제 가속과 감속에 따른 속도 변화가 반영되지 않았다. 또한 운전자 입력 장치와 차량 시뮬레이터, STM32 ECU가 독립적으로 동작하여 차량 상태 데이터를 통합 위험 판단에 활용하는 데 한계가 있었다.
원인 분석	CARLA 시뮬레이터의 차량 상태와 Logitech G29 입력을 CAN 메시지로 변환하는 소프트웨어 인터페이스가 충분히 연계되지 않았으며, PC에서 생성한 차량 데이터를 실제 CAN 물리 계층과 MCU 펌웨어를 거쳐 전달하는 종단 데이터 경로가 필요하였다.
해결 방안	CARLA의 동적 주행 속도와 G29의 조향, 브레이크, 가속 페달 입력을 CAN 프레임으로 변환하는 송신 프로그램을 구성하였다. CANable V2.0과 SN65HVD230을 통해 Classic CAN 500kbps 통신을 수행하고, STM32 FDCAN에서 CAN ID별 데이터를 수신 및 디코딩하도록 구현하였다. 이후 STM32의 차량 상태 정보를 UART 기반 인터페이스로 Raspberry Pi에 전달하여 통합 위험 판단에 활용할 수 있도록 연계하였다.
개선 결과	고정 속도 기반 시험 환경을 CARLA의 동적 주행 상태가 반영되는 임베디드 차량 통신 환경으로 확장하였다. 실기기 시험에서 차량 속도, 조향각, 브레이크 상태의 갱신을 확인하였으며, 120개 시험 샘플에서 CAN 데이터 유효성과 HMI 수신 상태를 검증하였다. 이를 통해 실차 없이도 시뮬레이션과 실제 ECU 하드웨어를 연계하는 시험 환경을 구축하였다.

장애요인 2 – STM32 FDCAN 통신 동기화 및 수신 신뢰성	
문제 현상	PC의 USB-CAN 장치와 STM32 ECU를 연결하는 과정에서 실제 CAN 프레임의 안정적인 수신과 차량 상태값의 정상적인 디코딩을 검증해야 했다. 특히 CAN 버스의 비트 타이밍 설정과 데이터 수신 중단 상황을 고려하지 않으면 잘못된 차량 상태가 위험 판단에 반영될 가능성이 있었다.
원인 분석	CAN 통신은 송신 장치와 수신 장치의 비트 타이밍 및 동기화 설정에 영향을 받는다. 또한 차량 속도, 브레이크, 조향각, 가속 페달이 서로 다른 CAN ID로 전달되므로 프레임 식별과 바이트 단위 디코딩이 정확해야 하며, 이전에 수신한 값이 통신 중단 후에도 유효한 값으로 유지되는 문제를 방지해야 했다.
해결 방안	STM32 HAL FDCAN을 활용하여 표준 11비트 CAN 프레임을 수신하고, CAN ID에 따라 데이터를 구분하여 차량 상태값으로 변환하였다. 통신 안정화를 위해 FDCAN 비트 타이밍을 점검하고 SJW 설정을 조정하였다. 또한 수신 프레임 카운터, 유효성 검사, 통신 오류 상태 및 수신 타임아웃을 확인하여 정상 수신과 통신 이상을 구분하도록 구성하였다.
개선 결과	초기 실기기 시험에서 유효 CAN 프레임 946건을 수신하였으며, 해당 시험 구간에서 invalid, lost, bus-off 카운터가 모두 0으로 확인되었다. 후속 시험에서는 CARLA 연동 차량 데이터의 수신 상태를 검증하고, CAN 송신 중단 시 차량 데이터 유효성이 해제되는 것을 확인하였다. 이를 통해 MCU 주변장치 설정부터 데이터 디코딩, 통신 이상 감지까지 차량 인터페이스의 신뢰성을 개선하였다.

장애요인 3- 전방 카메라 왜곡에 따른 객체 거리 및 TTC 추정 오차	
문제 현상	전방 카메라의 렌즈 왜곡과 촬영 구도 변화로 전방 카메라 영상에서 객체의 위치와 거리를 추정하는 과정에서 광각 렌즈의 왜곡과 카메라 설치 구도에 따라 추정값이 달라지는 문제가 발생하였다. 이러한 오차는 연속 관측값을 기반으로 계산하는 예상 충돌시간(TTC)의 안정성에도 영향을 줄 수 있었다.
원인 분석	카메라의 렌즈 왜곡과 촬영 각도 변화는 영상 좌표와 실제 주행 공간 사이의 대응 관계에 영향을 준다. 특히 객체 거리의 변화량으로 상대 접근 속도를 추정하는 경우, 거리값의 작은 오차나 검출 위치의 흔들림이 TTC 계산 결과에 반영될 수 있었다.
해결 방안	전방 카메라 보정 프로그램을 활용하여 렌즈 왜곡을 보정하고, 영상의 직선 잔차를 기준으로 보정 효과를 확인하였다. 객체 거리와 시간에 따른 거리 변화를 이용하여 상대 접근 속도 및 TTC를 계산하는 기능을 구현하였다. 또한 관측값의 유효성을 확인하고 접근하지 않는 객체에 대해 불필요한 TTC 경고가 발생하지 않도록 위험 판단 조건을 구성하였다.
개선 결과	카메라 보정 시험에서 직선 잔차를 1.229px에서 0.103px로 감소시켜 영상 왜곡을 개선하였다. 또한 객체 거리와 TTC 계산 기능을 Rule-based Risk Engine에 연계하여 전방 객체의 접근 상황을 위험 판단에 반영하였다. 다만 실제 거리와 TTC의 절대 오차 및 다양한 카메라 구도에서의 안정성은 추가 정량 검증이 필요한 항목으로 구분하였다.

장애요인 4 - 이기종 임베디드 장치 간 DDS 통신 및 데이터 정합성	
문제 현상	초기 Jetson 간 ROS 2 통신 시험에서 네트워크 연결은 가능했으나 DDS 토픽이 정상적으로 탐색되지 않는 문제가 발생하였다. 또한 외부 영상, 운전자 상태, 차량 정보는 장치별 처리 주기와 전달 지연이 서로 달라 수신 시점만으로 데이터를 통합하면 오래된 정보가 현재 상황으로 사용될 가능성이 있었다.
원인 분석	ROS 2 DDS는 네트워크 인터페이스와 피어 탐색 환경의 영향을 받으며, 서로 다른 임베디드 장치에서 동작하는 센서 노드는 데이터 생성 및 전송 주기가 일치하지 않는다. 따라서 정상적인 네트워크 연결뿐 아니라 장치 간 데이터 전달 상태와 각 데이터의 최신성을 별도로 관리해야 했다.
해결 방안	CycloneDDS의 네트워크 인터페이스와 피어 설정을 조정하고, 보드별 실행 환경과 서비스 런처를 정비하였다. 이후 DDS 메시지의 timestamp, sequence, TTL 및 heartbeat를 확인하도록 구성하였다. Raspberry Pi에서는 각 센서 데이터의 수신 상태와 유효성을 관리하고, 최신 유효 데이터를 기반으로 Situation Snapshot을 생성하도록 구현하였다.
개선 결과	통합 시험에서 세 보드의 서비스 6개가 정상 실행되고 heartbeat 4종이 연결되었으며, DDS 데이터 거부 0건을 기록하였다. 이후 전방 객체 인식 결과의 Raspberry Pi 전달도 확인하였다. 이를 통해 이기종 임베디드 장치의 통신 연결뿐 아니라 데이터 최신성과 상태를 관리하는 분산 소프트웨어 구조를 확보하였다.

장애요인 5- LLM 기반 위험 판단의 지연 문제와 경량화	
문제 현상	초기 LLM 기반 시스템을 시험하는 과정에서 위험 상황 해석 및 안내 문장 생성에 지연이 발생하였다. 특히 외부 API 요청 시간과 모델 응답 시간이 전체 처리시간에 영향을 미쳐, 목표로 설정한 종단 지연시간 200ms를 안정적으로 만족시키기 어려울 가능성이 확인되었다.
원인 분석	LLM 기반 처리 방식은 모델 추론뿐 아니라 네트워크 왕복, 입력 프롬프트 처리, 응답 생성 및 출력 파싱 등의 과정이 필요하다. CODRIVE는 여러 임베디드 장치에서 독립적으로 센서 데이터를 처리하고, DDS와 UART를 통해 Raspberry Pi로 전달하는 분산형 구조이다. 따라서 위험 판단 단계에서 추가적인 대기시간이 발생하면 전체 경고 지연이 증가할 수 있다.
해결 방안	실시간 경고 기능에서 자연어 생성을 필수 처리 단계로 사용하지 않도록 구조를 변경하였다. Situation Snapshot의 구조화된 특징값을 Rule-based Risk Engine 및 XGBoost 모델에 입력하여 위험도를 판단하고, Risk FSM을 통해 최종 경고 상태를 결정하도록 구성하였다. 또한 경고 출력은 사전 정의된 행동지침을 활용하여 일반적인 언어모델 응답 생성 과정과 분리하였다.
개선 결과	팀 측정 결과 XGBoost 모델 추론시간은 2ms로 확인되었다. 기존 LLM 기반 처리시간은 700ms, 개선된 위험 판단 및 경고 출력 처리시간은 95ms로 측정되었으며, 동일한 측정 범위에서 비교한 개선율은 87.7%이다. 전체 종단 지연시간은 개선 전 478ms, 개선 후 186ms이며, 최종 경고의 실시간성과 정상 출력 여부는 통합 시험 결과를 기준으로 평가한다.

5. 개발 결과물의 차별성

가. 기존 기술·제품 대비 차별성		
비교 항목	**기존 기술·제품**	**본 작품**
위험 인식 방식	전방 충돌, 차선 이탈, 운전자 부주의 등 개별 기능 중심의 위험 경고	외부 환경, 운전자 상태, CAN 차량 정보를 Situation Snapshot으로 통합하고 TTC와 차량 동역학을 결합하여 **다중 위험 요소의 연관성을 고려하는 상황 판단 기반 확보**
위험도 분석	객체 및 이벤트 감지, 거리 기반 위험 판단 등 기능별 분석	객체 거리와 TTC, CAN 속도 기반 가속도, 가속 페달 및 브레이크 상태를 함께 활용하여 충돌 접근 상황과 급가속 위험을 분석하는 복합 판단 기능 확보
시스템 구조	제품별로 중앙 집중형 또는 개별 ECU 구조 적용	Jetson, STM32, Raspberry Pi에 인식, 통신, 위험 판단 기능을 분산하고 ROS 2 DDS와 UART로 연계하여 실제 다중 보드 기반 분산 실행 및 시스템 통합 역량 확보
통신 안정성	시스템에 따라 개별적인 통신 오류 및 유효성 관리	Timestamp, TTL, heartbeat 및 CAN 수신 타임아웃을 적용하고 송신 중단 시 무효 상태 전환을 확인하여 데이터 최신성 관리와 통신 이상 식별 기능 확보
경고 제공 방식	개별 위험 발생에 따른 경고 제공	Rule-based 긴급 경고와 일반 자연어 안내를 분리하고 Risk FSM 및 hysteresis를 적용하여 긴급 경고의 처리 독립성과 위험 상태 안정화 구조 확보

나. 작품의 우수성	
①	**① 실제 CAN 하드웨어와 CARLA를 연계한 임베디드 차량 인터페이스** Logitech G29의 조향·가속·브레이크 입력을 CARLA 차량 제어와 연동하여 실제 운전자 조작에 따라 가상 차량이 주행하도록 구현하였다. 또한 CARLA에서 생성되는 차량 속도, 조향각, 브레이크 상태 등의 차량 정보를 CAN 메시지로 변환하고, CANable V2.0을 통해 실제 CAN Bus로 송신한 뒤 STM32 NUCLEO-H723ZG에서 FDCAN으로 수신·디코딩하였다. 이를 통해 실제 차량 없이도 운전자 입력부터 차량 주행, CAN 통신, 차량 상태 처리까지 연결되는 임베디드 차량 인터페이스를 구축하였다.
②	**② 다중 센서 융합과 차량 동역학 기반 복합 위험 판단** 외부 객체 인식 결과, 운전자 상태 및 CAN 차량 정보를 Situation Snapshot으로 통합하였다. 특히 객체 거리와 예상 충돌시간(TTC), 속도 변화 기반 가속도 추정 및 급가속 위험 감지 기능을 구현하여 단순한 객체 감지를 넘어 주행 상황에 따른 위험도를 분석한다. 이를 기반으로 여러 위험이 동시에 발생할 경우 최우선 경고를 결정한다.

③	**③ 통신 신뢰성과 사용자 안전을 고려한 분산형 임베디드 시스템** Jetson, STM32, Raspberry Pi 5에 기능을 분산하고 ROS 2 DDS와 CAN을 활용하여 장치 간 데이터를 연계하였다. 타임스탬프, TTL, heartbeat 기반 데이터 유효성 검사와 Risk FSM의 상태 안정화 기능을 적용하였으며, 전문가 컨설팅 의견을 반영해 긴급 경고를 일반 자연어 안내와 분리하였다. 이를 통해 통신 지연과 센서 상태 변화를 고려하면서 운전자에게 가장 시급한 행동지침을 우선 제공하도록 구성하였다. 실제 임베디드 하드웨어 시험에서 STM32 FDCAN의 유효 수신 946건과 통신 오류 카운터 0건을 확인하였다. 후속 CARLA 연동 시험에서는 동적 속도와 조향 및 브레이크 상태가 실제 CAN 버스를 거쳐 Raspberry Pi까지 전달되는 것을 검증하였으며, 120개 시험 샘플에서 정상적인 CAN 유효 상태와 HMI 수신 상태를 확인하였다. 또한 전방 카메라 렌즈 보정을 통해 직선 잔차를 1.229px에서 0.103px로 감소시켰다.

다. 실시간성 정량 평가 및 종단 지연 검증

① **① 실시간성 평가 목적**

본 시스템의 최종 성능은 개별 인지 모델의 처리속도뿐 아니라 위험 상황의 발생부터 운전자에게 최우선 경고가 전달되기까지의 종단 지연시간을 기준으로 평가하였다. 종단 지연시간의 설계 목표는 200ms 이내이며, 영상·음향 인지, 장치 간 데이터 전달, 데이터 통합, 위험 판단 및 경고 출력 과정을 측정 대상으로 설정하였다.

② **② 단계별 지연 측정 결과**

측정 구간	측정 결과
영상 AI 인지 처리	80ms
DDS 데이터 전달	14ms
Situation Snapshot 갱신 및 데이터 통합	0.20ms
XGBoost 모델 추론	2.35ms
Risk FSM 및 최종 경고 결정	0.13ms
HMI 경고 출력	90ms
전체 종단 지연	약 186ms

각 항목은 해당 처리 구간의 소요시간이며, 서로 병렬로 실행되는 인지·차량 데이터 처리 구간의 지연시간을 단순 합산하여 전체 종단 지연으로 계산하지 않는다.

③ **③ 개선 전후 지연시간 비교**

평가 항목	**초기 방식**	**개선 방식**
위험 판단·안내 처리시간	290ms	95ms
평균 종단 지연시간	478ms	186ms
P95 종단 지연시간	600ms	300ms
200ms 이내 경고 완료 비율	0%	87%
별도 LLM 시험(Qwen2.5-0.5B)	770ms	XGBoost 2ms 추론

	지연 개선율 | 87%p 증가 *** 지연 개선율** 개선율(%) = (개선 전 지연 – 개선 후 지연) / 개선 전 지연 × 100
④	**④ 평가 결과 분석** 모델 단위 시험에서는 XGBoost 추론시간 2ms를 확인하였다. 이를 바탕으로 실시간 위험 판단에 소요되는 모델 계산량을 경량화하였다. 통합 시험에서는 전체 지연시간과 95백분위 지연시간(P95)을 측정하여 평균적인 반응속도뿐 아니라 일부 상황에서 경고가 지연되는 현상도 함께 평가한다. 최종적으로 100회의 시험을 수행하고, 200ms 이내 경고 완료 비율 87%, 평균 종단 지연시간 186ms, P95 지연시간 약 300ms를 확인하여 실시간성 목표 달성 여부를 판단한다.

라. 복합 위험 시나리오 기반 통합 검증

①	**① 시나리오 구성 목적** 개별 센서 및 임베디드 장치의 정상 작동 여부를 확인하는 것에서 나아가, 여러 위험 요소가 동시에 발생하는 상황에서 시스템이 가장 시급한 위험을 선정하고 적절한 행동지침을 제공하는지 검증한다. 특히 단일 위험 인식 성능뿐 아니라 복합 위험 간 우선순위 결정, 통신 데이터 최신성, 위험 상태 전이 및 최종 경고 출력 과정을 통합적으로 평가한다.
②	**② 주요 검증 시나리오** 시나리오 | 입력 위험 상황 | 검증 항목 차선 변경 중 위험요소 발생 | 차선 변경 중 진행 경로에 차량 또는 보행자 등 위험 객체가 나타나는 상황 | 차선 변경 상태 및 외부 위험 객체 동시 인식 여부, Situation Snapshot 통합 여부, 충돌 위험도 상승 및 우선 경고 정상 출력 여부 악천후 환경 전방 위험 경고 | 우천·안개 등 시야가 저하된 환경에서 전방 차량 또는 보행자가 존재하는 상황 | 악천후 환경에서 외부 객체 인식 여부, 전방 위험 객체에 대한 위험도 계산 및 감속·주의 경고 정상 출력 여부 긴급차량 사이렌 + 적색 신호 복합 위험 | 긴급차량 사이렌이 감지되는 동시에 전방 신호등이 적색인 상황 | 음향·신호등 정보의 동시 통합 여부, 복수 Risk Event 생성 여부, 위험도 및 긴급성에 따른 최우선 경고 선정 여부 운전자 졸음·주의분산 | 내부 카메라에서 졸음 상태 또는 휴대폰 사용·전방 미주시가 감지되는 상황 | 운전자 상태 정상 인식 여부, 졸음·주의분산에 따른 위험도 상승 여부, 외부 위험과 결합 시 경고 단계 및 우선순위 정상 변경 여부
③	**③ 시나리오 시험 과정** 1. CARLA와 센서 입력을 통해 복합 위험 상황을 구성한다. 2. Jetson에서 외부 환경·운전자 상태·음향 인식 결과를 생성한다. 3. STM32를 통해 차량 속도, 조향각 및 브레이크 정보를 전달한다. 4. Raspberry Pi에서 최신 유효 데이터를 Situation Snapshot으로 통합한다. 5. Rule-based Risk Engine, XGBoost 및 Risk FSM으로 위험 우선순위를 결정한다. 6. HMI·TTS의 최종 경고 내용과 출력 지연시간을 기록한다.

④

④ 시나리오별 평가 결과

시나리오	시험 횟수	정상 우선순위 선정
S1. 차선 변경 중 위험요소 발생	10회 이상	정상 동작 확인
S2. 악천후 환경 전방 위험 경고	10회 이상	정상 동작 확인
S3. 사이렌 + 적색 신호 복합 위험	10회 이상	정상 동작 확인
S4. 운전자 졸음 및 주의분산	10회 이상	정상 동작 확인

시험 결과를 통해 복합 위험 발생 시 가장 중요한 행동지침이 정상적으로 선택되는지 확인하고, 경고 우선순위 오류 및 지연 초과 사례를 분석한다. 또한 위험 상황 해제 후 Risk FSM의 정상 상태 복귀와 반복적인 경고 발생 여부를 검증하여 시스템의 안정성을 평가한다.

6. 개발 결과물의 파급력 및 기대효과

판매 가치	CODRIVE는 차량 외부 위험과 운전자 상태, CAN 기반 차량 정보를 통합하여 최우선 행동지침을 제공하는 운전 보조 솔루션이다. 기존 ADAS의 개별 경고를 복합 상황 중심의 안내로 보완하여 운전자의 정보 해석 부담을 줄이는 차별화된 가치를 제공한다. 또한 차량 제어에 직접 개입하지 않는 경고 중심 구조를 바탕으로, 기존 차량용 보조 장치와 운전 교육용 시스템으로의 제품화 가능성을 갖는다.
시장성	국가데이터처의 「2025 고령자 통계」에 따르면 국내 65세 이상 인구는 전체의 20.3%에 달하며, 국회입법조사처 자료에서는 2023년 고령 운전자 교통사고가 39,614건으로 집계되었다. 이는 고령 운전자와 초보 운전자 등 다양한 사용자를 위한 안전 지원 기술의 필요성을 보여준다. 또한 Euro NCAP의 2026년 안전 평가 체계에서는 운전자 모니터링과 HMI 사용성을 강조하고 있다. 이러한 변화에 따라 CODRIVE의 운전자 상태 기반 복합 위험 판단과 직관적인 행동 안내 기술은 차량 안전 보조 서비스 및 운전 교육 분야에서 활용 가능성이 있다.
활용성	CODRIVE는 Jetson의 AI 인식, STM32의 CAN 차량 통신, Raspberry Pi의 위험 판단 및 HMI 기능을 분산형 임베디드 구조로 구현하였다. 특히 CARLA의 동적 주행 정보와 실제 CAN 물리 계층을 연결하여 실차 없이도 차량 속도, 조향, 브레이크 및 가속 페달 정보를 활용한 위험 상황을 반복 시험할 수 있다. 따라서 운전자 안전 보조 기술뿐 아니라 ECU 통신, 임베디드 소프트웨어 개발, 차량 위험 알고리즘 및 운전 교육 시뮬레이터의 시험 환경으로 활용할 수 있다.
발전 가능성	구현된 TTC 기반 충돌 위험 판단과 차량 동역학 기반 급가속 감지 기능을 발전시켜 보행자 접근, 운전자 부주의, 차량 이상 주행 등 다양한 복합 위험 상황에 대응할 수 있다. 또한 실제 주행 데이터 기반 모델 개선과 센서별 오탐 분석, 종단 지연시간 측정을 통해 실시간성과 신뢰성을 높일 수 있다. 향후 차량 인터페이스의 호환성과 시스템 경량화를 개선하고, 운전자 상태에 따라 경고 민감도를 조절하는 방향으로 확장할 수 있다.

7. 개발 일정 및 업무 분장

가. 업무 분장			
No	구분	성명	담당 업무
1	팀장	서유나	**위험도 판단 및 HMI 개발**: Situation Snapshot 기반 Rule-based 위험도 계산 로직 설계 및 구현, 위험 요소별 가중치 및 경고 우선순위 기준 정의, XGBoost 위험도 분류 모델 학습·평가·튜닝, HMI 화면 구성 및 최종 위험 경고 출력 기능 개발
2	팀원	권현준	**전체 시스템 하드웨어 설계 및 통합 환경 구축**: 배선 구조 설계, Jetson·STM32 NUCLEO-H723ZG·Raspberry Pi·CANable·Logitech G29 및 센서의 전체 하드웨어 구성 설계, 각 임베디드 장치 개발환경 구축, 하드웨어 배치 및 시스템 구동 환경 구성
3	팀원	백건하	**내부 카메라 인식 및 전체 시스템 통합**: 내부 카메라 기반 운전자 상태 인식 모듈 개발, 운전자 상태 데이터 정형화 및 ROS2 DDS 연동, 각 인지·통신·위험 판단 모듈의 End-to-End 시스템 통합, 복합 위험 시나리오 통합 테스트 및 오류 수정
4	팀원	유용준	**음향 인식 모델 개발**: 외부 마이크 기반 주행 환경 음향 데이터 수집 및 전처리, 긴급차량 사이렌·경적 등 위험 음향 구분 모델 개발, 음향 데이터 특징 추출 및 학습 데이터셋 구성, 모델 학습·평가 및 추론 성능 개선
5	팀원	황예은	**CAN 데이터 통신 및 외부 카메라 인식 개발**: Logitech G29 조향·가속·브레이크 입력과 CARLA 차량 제어 연동, CANable V2.0 기반 Classic CAN 500 kbps 메시지 송신, Jetson 기반 외부 카메라 차량·보행자·신호등·차선 인식 모듈 개발

나. 개발 일정																				
No	추진 내용	6월				7월				8월				9월				10월			
		1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
1	시스템 아키텍처 구상																				
2	하드웨어 구축																				
3	데이터 인터페이스 디자인																				
4	인지 모듈 개발																				
5	Situation Fusion 개발																				
6	리스크 관리																				
7	Hardware 통합																				
8	LLM, HMI & System 통합																				
9	테스트 및 최적화																				
10	최종 시연																				